

Sistema de Videovigilância - SafeCam

Licenciatura em Engenharia Informática

João Martins Tendeiro

Miguel Francisco Lopes

Leiria, junho de 2025

Sistema de Videovigilância - SafeCam

Licenciatura em Engenharia Informática

João Martins Tendeiro

Miguel Francisco Lopes

Trabalho de Projeto da unidade curricular de Engenharia de Sistemas e Serviços realizado sob a
orientação do Professor Doutor António Pereira

Leiria, junho de 2025

Resumo

O presente relatório apresenta o desenvolvimento do SafeCam, um sistema de videovigilância inteligente concebido no âmbito da unidade curricular de Engenharia de Sistemas e Serviços. O objetivo deste projeto é desenvolver uma solução robusta e acessível para a vigilância contínua de infraestruturas, capaz de identificar automaticamente pessoas e veículos em tempo real, recorrendo a um modelo de inteligência artificial.

A solução proposta baseia-se no modelo YOLO11n, conhecido pela sua leveza e elevada eficiência em tarefas de inferência local. O sistema capta vídeo a partir de uma câmara (integrada ou via USB), processa os frames localmente e envia automaticamente alertas por e-mail sempre que são detetados objetos relevantes. Para complementar o sistema, foi criada uma interface web intuitiva que permite acompanhar o vídeo em tempo real, aceder facilmente às gravações e receber notificações visuais sempre que são detetados eventos relevantes.

O protótipo foi testado em diversos cenários, incluindo simulações com vídeos pré-gravados e funcionamento em tempo real com webcam. Os resultados demonstram a precisão da deteção, a capacidade de contagem por zonas do enquadramento e a fiabilidade do sistema de alertas. O SafeCam revelou-se uma solução robusta, modular e escalável.

Palavras-chave: videovigilância, YOLO11n, inteligência artificial, alertas.

Abstract

This report presents the development of SafeCam, an intelligent video surveillance system designed as part of the Systems and Services Engineering course. The goal of this project is to develop a robust and accessible solution for continuous environmental monitoring, capable of automatically identifying people and vehicles in real time using an artificial intelligence model.

The proposed solution is based on the YOLO11n model, known for its lightweight structure and high efficiency in local inference tasks. The system captures video from an integrated or USB-connected camera, processes the frames locally, and automatically sends email alerts whenever relevant objects are detected. To complement the system, an intuitive web interface was developed, allowing users to watch the live video stream, easily access recordings, and receive visual notifications whenever relevant events are detected.

The prototype was tested in various scenarios, including simulations with pre-recorded videos and real-time webcam operation. The results demonstrate the accuracy of the detection, the capability to count objects in specific zones of the frame, and the reliability of the alert system. SafeCam proved to be a robust, modular, and scalable solution. Keywords: video surveillance, YOLO11n, artificial intelligence, alerts.

Keywords: video surveillance, YOLO11n, artificial intelligence, alerts.

Índice

Resumo	i
Abstract	ii
Lista de Figuras	v
Lista de Tabelas	vi
Lista de siglas e acrónimos	vii
1 Introdução	1
2 Estado da arte	3
2.1 Soluções existentes	3
2.2 Comparação de Modelos	6
2.3 Ultralytics YOLO11	7
2.3.1 Características	8
2.3.2 Capacidades Funcionais do YOLO11	9
2.3.3 Desempenho dos Modelos YOLO11	9
2.4 Escolha do Modelo	10
3 Cenário de Implementação	12
3.1 Protótipo	13
3.2 Tecnologias	13
3.2.1 Linguagem e Frameworks	14
3.2.2 Visão Computacional	14
3.2.3 Sistema de Notificações	14
3.2.4 Gravação de Vídeo e Armazenamento	15
3.3 Módulos	15

3.4	Funcionalidades	16
3.5	Interface Web	16
3.6	Código desenvolvido	18
3.6.1	Envio Automático de Alertas por Email	18
3.6.2	Classificação e Anotação de Objetos Detetados	19
3.7	Gravação Assíncrona de Vídeo Detetado	22
4	Testes e Resultados	24
4.1	Cenários de Teste	24
4.1.1	Cenário de teste A	24
4.1.2	Cenário de teste B	27
4.1.3	Cenário de teste C	28
5	Mais Valias	31
6	Análise crítica e proposta de melhorias	32
7	Conclusão	33
	Bibliografia	34
	Anexos	35

Lista de Figuras

1	Frigate	4
2	DeepStack	5
3	Blue Iris	5
4	Modelos YOLO e outros modelos	8
5	Arquitetura Lógica	13
6	Página Principal	17
7	Página Capturas	18
8	Função para envio de Alertas por Email	19
9	Código responsável pela classificação de pessoas e carros detetados	20
10	Código responsável pela definição e visualização das zonas de deteção	21
11	Código responsável pela lógica de emissão e fim de alertas	22
12	Classe responsável pela gravação assíncrona dos vídeos detetados	23
13	Cenário A	25
14	Deteção de pessoas e alertas na interface WEB	26
15	Alertas recebidos no email	26
16	Vídeo gravado	26
17	Cenário B	27
18	Deteção de pessoas e carros	28
19	Cenário C	29
20	Deteção de pessoas	30
21	Deteção de carros	30

Lista de Tabelas

1	Comparação entre modelos de detecção de objetos	7
2	Comparação entre os modelos YOLO11	10
3	Resumo dos Cenários de Teste	24

Lista de siglas e acrónimos

API Application Programming Interface. 4

CNN Convolutional Neural Network. 6, 7

COCO Common Objects in Context. 8, 10, 14

CPU Central Processing Unit. 3, 4, 6, 10, 12, 14

CSS Cascading Style Sheets. 14, 16

FLOPs Floating-Point Operations Per Second. 10

FPS Frames Per Second. 6, 7

GPU Graphics Processing Unit. 4, 6, 8–10, 14

HTML Hypertext Markup Language. 14–16

JPEG Joint Photographic Experts Group. 14

mAP Mean Average Precision. 6–10

MB Megabyte. 6, 7

MIME Multipurpose Internet Mail Extensions. 15

NVR Network Video Recorder. 3

OBB Oriented Bounding Box. 9

ONNX Open Neural Network Exchange. 10

REST Representational State Transfer. 4

SMTP Simple Mail Transfer Protocol. 12–14, 18

TLS Transport Layer Security. 18

TPU Tensor Processing Unit. 3

USB Universal Serial Bus. i, 12, 13

WEB World Wide Web. v, 12–16, 25–28

YOLO You Only Look Once. v, 3, 7, 8, 18, 19

1 Introdução

Atualmente, muitas empresas enfrentam desafios na segurança das suas infraestruturas devido ao aumento de roubos e comportamentos suspeitos. A falta de um sistema de videovigilância eficiente limita a monitorização em tempo real e a resposta rápida a incidentes. A implementação de um serviço de videovigilância eficiente e seguro tornou-se essencial para garantir a proteção dos bens e a segurança dos utilizadores, permitindo a monitorização contínua e o armazenamento seguro das gravações, acessíveis apenas a pessoas autorizadas.

A nossa solução de Vídeo Vigilância “SafeCam” é uma solução segura para capturar, transmitir e armazenar vídeos, garantindo a proteção dos dados e a monitorização contínua de instalações. Este serviço oferece segurança dos dados, monitorização em tempo real através de uma interface web, escalável e armazenamento seguro. É destinado a empresas, lojas, escolas e câmaras municipais, que precisam de monitorização contínua e segura das suas instalações. Atualmente, as soluções de videovigilância no mercado não permitem monitorizar e controlar em tempo real o vídeo de forma integrada e segura numa Cloud privada, dependendo geralmente de servidores de terceiros.

Este relatório apresenta a conclusão do segundo trabalho prático da unidade curricular de Engenharia de Sistemas e Serviços, centrando-se na implementação do protótipo funcional do SafeCam. A solução contempla funcionalidades essenciais, como a monitorização em tempo real, gestão remota e envio automático de alertas por e-mail com base na deteção de pessoas e veículos utilizando inteligência artificial.

Os principais objetivos do projeto SafeCam, que orientaram todo o desenvolvimento do protótipo funcional, centram-se em:

- Desenvolver uma solução de videovigilância segura, que permita a transmissão, captura e armazenamento de vídeo.
- Implementar deteção em tempo real de objetos relevantes, nomeadamente pessoas e carros.

- Garantir que o sistema disponibiliza uma interface web intuitiva para monitorização remota e gestão dos alertas.
- Permitir o envio automático de notificações por e-mail quando forem detetados eventos relevantes, promovendo uma resposta rápida.
- Desenvolver uma arquitetura escalável que possa ser integrada numa cloud privada, preservando a segurança e privacidade dos dados.

O relatório está estruturado em cinco capítulos principais: o Capítulo 1 apresenta a introdução ao projeto e a motivação para o desenvolvimento do SafeCam; o Capítulo 2 corresponde ao estado da arte, onde são analisadas soluções existentes de videovigilância e comparação de diversos modelos de deteção de objetos, com especial destaque para o YOLO11; o Capítulo 3 descreve o desenvolvimento do protótipo, incluindo as tecnologias utilizadas, arquitetura do sistema, funcionalidades e implementação prática; o Capítulo 4 foca-se nos testes realizados e respetivos resultados obtidos em diferentes cenários de validação; o Capítulo 5 apresenta as possíveis melhorias futuras do sistema; por fim, o Capítulo 6 apresenta a conclusão do trabalho.

2 Estado da arte

Nesta secção são analisadas as principais soluções existentes de videovigilância inteligente, como o **Frigate**, **DeepStack** e **Blue Iris**, destacando as suas características, vantagens e limitações. Em seguida é apresentada uma revisão das principais arquiteturas de deteção de objetos, com foco nas variantes YOLO e em modelos concorrentes, comparando-os quanto a precisão, velocidade e requisitos computacionais. Por fim, apresenta-se o Ultralytics YOLO11 e a justificativa da seleção do YOLO11n para o protótipo.

2.1 Soluções existentes

Atualmente, existem várias soluções disponíveis para a deteção de objetos e videovigilância inteligente, que combinam algoritmos de visão computacional, com funcionalidades de gravação e automação. Entre as mais relevantes encontram-se o **Frigate**, o **DeepStack** e o **Blue Iris**, cada uma com características distintas em termos de arquitetura, desempenho e uso.

- **Frigate**: Plataforma de NVR open-source desenvolvida para funcionar de forma eficiente com o *Google Coral TPU*, utiliza modelos quantizados em *TensorFlow Lite*. O Frigate destaca-se pela sua integração nativa com o *Home Assistant*, deteção de objetos baseada em zonas configuráveis e baixa utilização de CPU. Os modelos utilizados, embora otimizados para velocidade, comprometem parcialmente a precisão em relação a arquiteturas mais modernas como o YOLO11.



Figura 1: Frigate

- **DeepStack:** Solução baseada em APIs REST para detecção de objetos, reconhecimento facial e classificação de imagens. Utiliza modelos predefinidos baseados em YOLOv3, oferecendo boa precisão com suporte limitado à personalização. Funciona principalmente via CPU ou GPU, mas com desempenho inferior em tempo real quando comparado com soluções que utilizam aceleração por hardware dedicado.

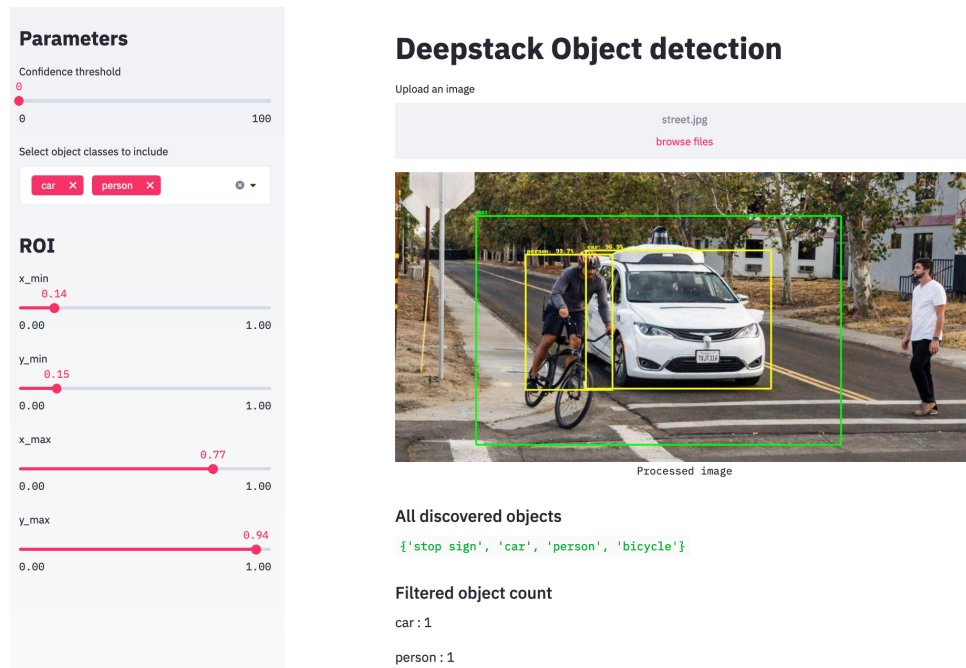


Figura 2: DeepStack

- **Blue Iris:** Software proprietário para videovigilância com suporte para múltiplas câmaras, detecção de movimento e integração com ferramentas externas como o DeepStack. Oferece uma interface gráfica, mas depende de recursos computacionais consideráveis quando é integrado com inteligência artificial, além de estar limitado ao sistema operativo Windows.

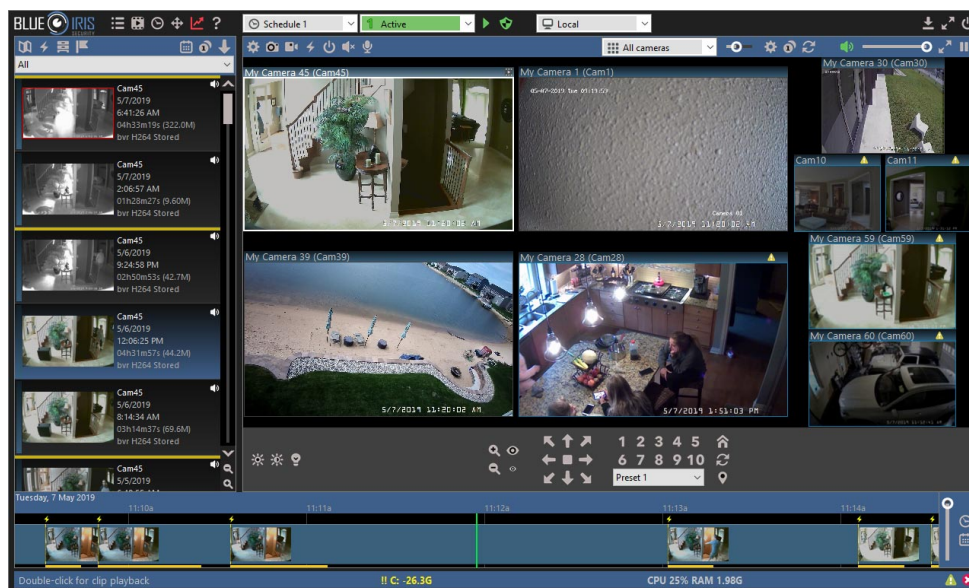


Figura 3: Blue Iris

Estas soluções oferecem soluções completas, com foco na detecção de eventos, mas limitam a flexibilidade em termos de escolha e personalização de modelos. Neste contexto, a utilização direta de modelos mais modernos como o YOLO11n permite um maior controlo sobre os parâmetros de inferência, personalização de classes e adaptação a diferentes requisitos de hardware e software, sendo, por isso, uma opção mais adequada para o desenvolvimento e otimização de protótipos personalizados.

2.2 Comparação de Modelos

A análise dos diferentes modelos de detecção de objetos, atualmente utilizados na área da visão computacional, considera diversos critérios relevantes, nomeadamente a arquitetura do modelo, a precisão média (mAP@0.5), a velocidade de inferência (FPS), a dimensão do modelo e os contextos de aplicação recomendados. Modelos mais leves, como o YOLOv8n, são particularmente adequados para sistemas em tempo real com recursos computacionais limitados, como equipamentos que dispõem apenas de CPU, embora também possam beneficiar de aceleração por GPU, garantindo um desempenho mais consistente em cenários com maiores exigências.

Por outro lado, modelos mais pesados, como o Faster R-CNN ou o DETR, oferecem níveis de precisão superiores, mas com maior exigência computacional, sendo mais apropriados para processamento *offline* ou em servidores com GPU (como podemos observar na tabela 1).

Modelo	Arquitetura	mAP@0.5	FPS	Tamanho	Ideal para
YOLOv8n / YOLO11n	One-stage (CNN leve)	30–35%	120–180	~5 MB	Edge AI, CPUs, tempo real
YOLOv8m	One-stage (CNN média)	45–50%	50–70	~20 MB	Equilíbrio entre precisão e velocidade
YOLOv5s	One-stage (CSPDarknet)	35–40%	90–120	~14 MB	Equipamentos leves, robustos
YOLOv4-tiny	One-stage (Darknet)	30–35%	100+	~6 MB	Drones

Modelo	Arquitetura	mAP@0.5	FPS	Tamanho	Ideal para
EfficientDet-D0	One-stage (EfficientNet)	35–40%	30–40	~15 MB	Edge/cloud híbrido, aplicações móveis
SSD MobileNet	One-stage (MobileNetV2)	25–35%	40–60	~10 MB	Telemóveis, baixo consumo
Faster R-CNN	Two-stage (ResNet-50/101)	50–60%	5–15	>150 MB	Alta precisão, processamento offline
DETR	Transformer- based (ViT)	42–50%	3–10	>180 MB	Projetos de investigação
RetinaNet	One-stage (ResNet + Focal Loss)	35–45%	15–30	>100 MB	Deteção com classes desbalanceadas

Tabela 1: Comparação entre modelos de detecção de objetos

A comparação, apresentada na Tabela 1, tem como objetivo apoiar a escolha do modelo mais adequado em função dos requisitos específicos de cada aplicação, nomeadamente no equilíbrio entre desempenho, precisão e eficiência.

2.3 Ultralytics YOLO11

O YOLO11 é uma das versões mais recentes dos detetores de objetos em tempo real da série Ultralytics YOLO, elevando os padrões de precisão, velocidade e eficiência. Com base nos avanços impressionantes das versões anteriores do YOLO, o YOLO11 introduz melhorias significativas na arquitetura e nos métodos de treino, tornando-se uma escolha versátil para diversos tipos de tarefas de visão computacional.

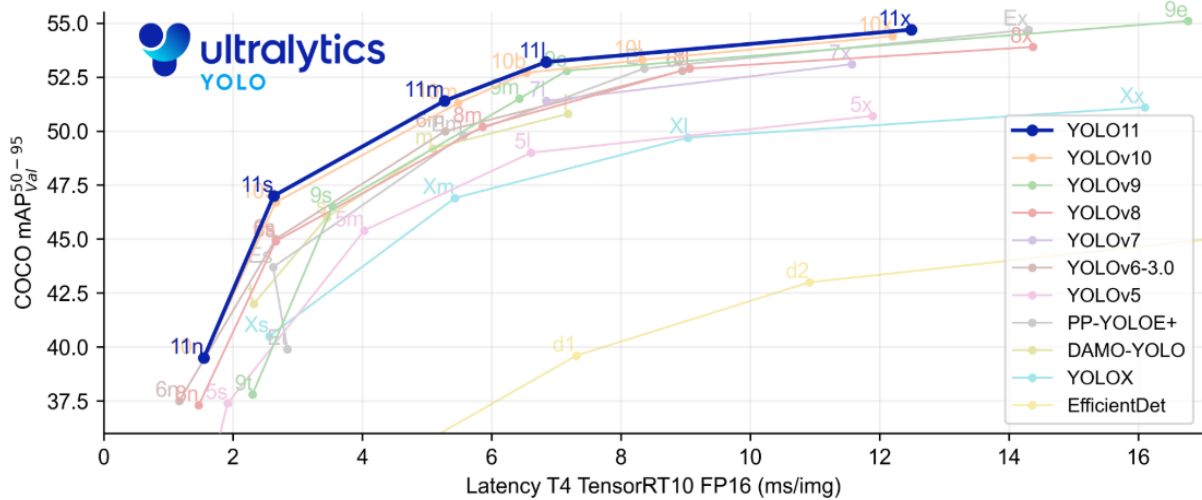


Figura 4: Modelos YOLO e outros modelos

Como se pode observar na Figura 4, o gráfico apresenta a relação entre a precisão (mAP) e a latência de inferência (em milissegundos), medida com TensorRT numa GPU NVIDIA T4, para os diversos modelos YOLO e outros detetores de objetos. O YOLO11 destaca-se em relação aos outros modelos pelo equilíbrio entre precisão e velocidade de inferência, sendo especialmente indicado para aplicações em tempo real que requerem elevada eficiência computacional.

2.3.1 Características

- **Extração de características melhorada:** o YOLO11 utiliza uma arquitetura aprimorada de *backbone and neck*, melhorando a capacidade de extração de características e permitindo uma deteção de objetos mais precisa, mesmo em tarefas complexas.
- **Otimizado para eficiência e velocidade:** apresenta um desenho arquitetónico refinado e um *pipeline* de treino otimizado, proporcionando velocidades de processamento mais elevadas sem comprometer a precisão.
- **Maior precisão com menos parâmetros:** graças a avanços na conceção do modelo, o YOLO11m alcança uma precisão média (mAP) superior no conjunto de dados COCO, utilizando menos 22% de parâmetros comparativamente ao YOLOv8m, o que o torna mais

eficiente em termos computacionais.

- **Adaptabilidade a diferentes ambientes:** o YOLO11 pode ser facilmente implementado em diversos ambientes, incluindo equipamentos *edge*, plataformas na *cloud* e sistemas com GPUs NVIDIA, oferecendo elevada flexibilidade.
- **Suporte a múltiplas tarefas de visão computacional:** o YOLO11 foi concebido para responder a muitos desafios, como detecção de objetos, segmentação por instância, classificação de imagens, estimativa de pose e detecção orientada de objetos (OBB).

2.3.2 Capacidades Funcionais do YOLO11

- **Deteção de objetos:** identificação e localização de objetos presentes numa imagem, através de *bounding boxes*.
- **Segmentação de instâncias:** deteção precisa de objetos com contornos delineados individualmente, permitindo separar objetos da mesma classe.
- **Classificação de imagens:** categorização de imagens em classes previamente definidas com base nas suas características visuais.
- **Estimativa de pose:** deteção e seguimento de pontos-chave (por exemplo, articulações) no corpo humano, essencial para análise de movimento.
- **Deteção orientada de objetos (OBB):** deteção de objetos com orientação angular, através de caixas delimitadoras, para maior precisão em cenários complexos.

2.3.3 Desempenho dos Modelos YOLO11

Modelo	Tamanho (px)	mAPval 50–95	Vel. CPU ONNX (ms)	Vel. T4 TensorRT10 (ms)	Parâmetros (M)	FLOPs (B)
YOLO11n	640	39.5	56.1 ± 0.8	1.5 ± 0.0	2.6	6.5
YOLO11s	640	47.0	90.0 ± 1.2	2.5 ± 0.0	9.4	21.5

Modelo	Tamanho (px)	mAPval 50–95	Vel. CPU ONNX (ms)	Vel. T4 TensorRT10 (ms)	Parâmetros (M)	FLOPs (B)
YOLO11m	640	51.5	183.2 ± 2.0	4.7 ± 0.1	20.1	68.0
YOLO11l	640	53.4	238.6 ± 1.4	6.2 ± 0.1	25.3	86.9
YOLO11x	640	54.7	462.8 ± 6.7	11.3 ± 0.2	56.9	194.9

Tabela 2: Comparação entre os modelos YOLO11

A tabela 2 apresenta uma comparação entre as diferentes variantes do modelo YOLO11 (YOLO11n, YOLO11s, YOLO11m, YOLO11l e YOLO11x), utilizando métricas baseadas no conjunto de dados COCO. São analisadas características como a resolução da imagem de entrada, a precisão média (mAPval 50–95), o tempo de inferência em CPU e GPU (TensorRT), o número de parâmetros e a complexidade computacional (FLOPs).

2.4 Escolha do Modelo

Com base na análise comparativa apresentada, a escolha recaiu sobre o modelo **YOLO11n**, por este proporcionar um compromisso otimizado entre desempenho e eficiência. Conforme indicado na tabela 2, o YOLO11n apresenta:

- **Baixa latência**, particularmente relevante para aplicações em tempo real;
- **Número reduzido de parâmetros** (2.6M), o que resulta em menor uso de memória e processamento;
- **Boa precisão** para tarefas de detecção de objetos, com um mAP de 39.5% no COCO;
- **Alta velocidade de execução**, tanto em CPU (56.1 ms) como em GPU (1.5 ms), conforme medições com ONNX e TensorRT;
- **Tamanho compacto**, facilitando a sua implementação em sistemas com recursos computacionais limitados.

Embora existam variantes do YOLO11 com maior precisão (e.g., YOLO11m ou YOLO11x), estas exigem significativamente mais recursos, comprometendo a viabilidade de execução em sistemas convencionais.

Considerando que o presente protótipo será implementado em computadores pessoais, a utilização do YOLO11n revela-se especialmente adequada, maximizando o desempenho sem comprometer a usabilidade da solução.

3 Cenário de Implementação

A solução foi desenvolvida e testada localmente, nos computadores pessoais, sem recurso a servidores externos ou a plataformas de cloud. Esta abordagem permitiu validar a viabilidade da solução em ambiente real, com recursos limitados, demonstrando que o sistema pode funcionar de forma autónoma em equipamentos convencionais.

A captura de vídeo foi realizada através de webcams integradas ou ligadas por USB, sendo o processamento dos *frames* efetuado em tempo real na CPU do computador utilizado.

O servidor WEB e a interface de visualização funcionaram localmente, através da framework Flask. Esta configuração permitiu a visualização em tempo real da transmissão de vídeo e a gestão das capturas armazenadas. O envio de alertas por email via protocolo SMTP, foi igualmente configurado localmente, com autenticação dos servidores do Gmail.

A gravação e organização dos vídeos foi feita no sistema de ficheiros local, permitindo ao utilizador visualizar e eliminar os vídeos diretamente através da interface. Todas as bibliotecas e dependências necessárias foram instaladas e geridas localmente, o que assegura a portabilidade do sistema para outros equipamentos sem necessidade de ligação à Internet.

3.1 Protótipo

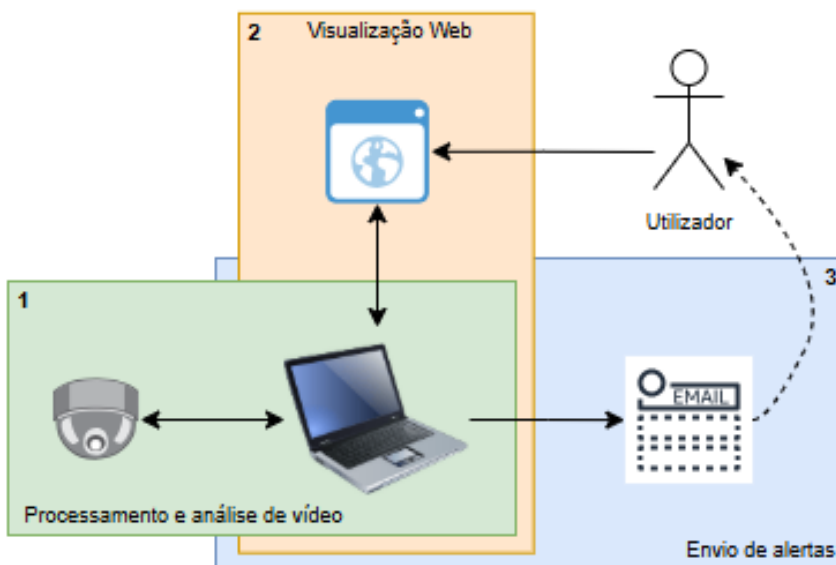


Figura 5: Arquitetura Lógica

No protótipo desenvolvido, um computador executa o serviço responsável por capturar vídeo a partir de uma câmara integrada ou externa (via USB). Os frames são processados localmente com o modelo YOLO11n, e os resultados são apresentados numa interface WEB. Sempre que é detetada uma pessoa ou um carro, o sistema envia automaticamente um alerta por email (via SMTP) para o utilizador (**fig. 5**).

3.2 Tecnologias

O desenvolvimento do sistema de deteção, notificação e monitorização em tempo real foi suportado por um conjunto de tecnologias devidamente integradas, que abrangem desde a visão computacional até ao envio automatizado de alertas.

3.2.1 Linguagem e Frameworks

A linguagem utilizada foi o **Python 3.10**, escolhida pela sua simplicidade e pela vasta oferta de bibliotecas orientadas à visão computacional e ao desenvolvimento de aplicações WEB.

O **Flask** foi utilizado como microframework *backend*, proporcionando uma base para a criação de rotas, gestão do servidor WEB e interação com a interface do utilizador. A renderização das páginas "index.html" (visualização em tempo real) e "capturas.html" (gestão de capturas) é feita diretamente a partir das rotas do Flask.

O **Flask-SocketIO** foi integrado para possibilitar comunicação bidirecional em tempo real com o *frontend*. Esta tecnologia permite o envio de alertas instantâneos para a interface sempre que um evento é detetado, sem necessidade de atualizar manualmente a página.

A interface WEB foi desenvolvida com HTML, CSS e JavaScript.

3.2.2 Visão Computacional

A biblioteca **OpenCV** é utilizada para captura de vídeo, leitura de ficheiros pré-gravados e leitura de frames em tempo real. Também é responsável pela codificação de imagens JPEG, sobreposição de anotações visuais (caixas delimitadoras e percentagem) e divisão da imagem em zonas verticais.

O modelo de deteção é o **YOLO11n**, disponibilizado pela biblioteca Ultralytics. Este modelo foi selecionado com base na análise feita no Capítulo Estado da arte, devido ao seu bom desempenho em tarefas de deteção em tempo real. Trata-se de um modelo leve e eficiente, pré-treinado no conjunto de dados COCO, com suporte para inferência em CPU e GPU.

3.2.3 Sistema de Notificações

O sistema de notificações é baseado em SMTP, que utiliza autenticação com o servidor do gmail. Sempre que uma pessoa ou carro é detetado, o sistema gera um alerta que é enviado por email, seguindo posteriormente o vídeo desse alerta.

Este envio é feito de forma automática, utilizando a biblioteca `smtplib` e estruturas MIME para permitir o envio de anexos.

3.2.4 Gravação de Vídeo e Armazenamento

A gravação dos eventos detetados foi implementada utilizando a biblioteca OpenCV em conjunto com `threading` do Python, permitindo processar e armazenar os vídeos de forma assíncrona. Os vídeos são codificados com o codec VP8 e guardados no formato `.webm`.

O armazenamento local é organizado em pastas acessíveis via servidor WEB, sendo possível visualizar e gerir os vídeos diretamente através da interface. As funcionalidades de visualização e eliminação foram desenvolvidas com recurso a HTML, JavaScript e rotas Flask.

3.3 Módulos

- **Módulo de Processamento e Análise de Vídeo¹:** Responsável por capturar os frames da webcam ou vídeo e realizar a deteção de objetos em tempo real (pessoas e carros) utilizando o modelo YOLOv11n da Ultralytics. Sempre que um objeto é detetado, inicia-se automaticamente a gravação, que termina após 2 segundos de ausência. Os vídeos gravados são armazenados localmente.
- **Módulo de Visualização WEB²:** Permite ao utilizador visualizar em tempo real a transmissão de vídeo da webcam ou vídeo e ver os alertas que são emitidos ao ser detetado um objeto. O utilizador pode também aceder às capturas gravadas, visualizá-las e eliminá-las através da interface WEB.
- **Módulo de Envio de Alertas³:** Responsável por notificar o utilizador sempre que um objeto é detetado, através de mensagens em tempo real enviadas para o frontend via SocketIO e por email, incluindo os vídeos gravados.

3.4 Funcionalidades

- Visualização em tempo real da transmissão da webcam ou de um vídeo pré-gravado.
- Detecção automática de pessoas e veículos, com destaque visual (bounding boxes).
- Contagem em tempo real do número de pessoas e veículos detetados.
- Alertas visuais em tempo real na interface WEB.
- Envio automático de emails com mensagem de alerta.
- Gravação e envio do vídeo da ocorrência por email após a detecção.
- Galeria de capturas com opção de visualização e remoção.

3.5 Interface Web

A interface WEB foi desenvolvida em Python, com recurso às bibliotecas Flask e Flask-SocketIO. A parte visual da interface foi construída com HTML, CSS e JavaScript.

A página principal da aplicação é responsável por apresentar a transmissão em tempo real de vídeo em direto, proveniente da webcam ou de um vídeo pré-gravado. Além disso, integra um painel de alertas dinâmico, que exibe notificações sempre que é detetada uma pessoa ou um carro.

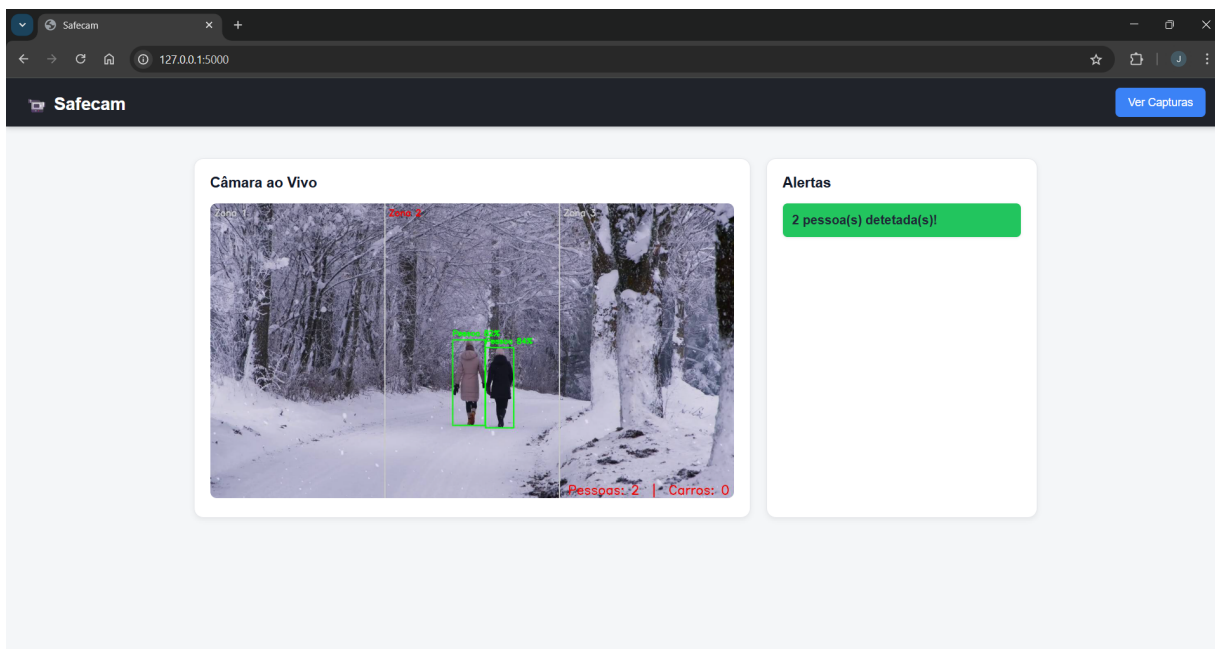


Figura 6: Página Principal

Na página "Capturas" o utilizador pode consultar todos os segmentos de vídeo gravados automaticamente. A interface permite não só visualizar os vídeos diretamente no navegador, como também eliminá-los de forma simples através de um botão de remoção associado a cada gravação. Esta funcionalidade garante uma gestão prática e autónoma dos conteúdos registados, sem necessidade de aceder ao servidor.

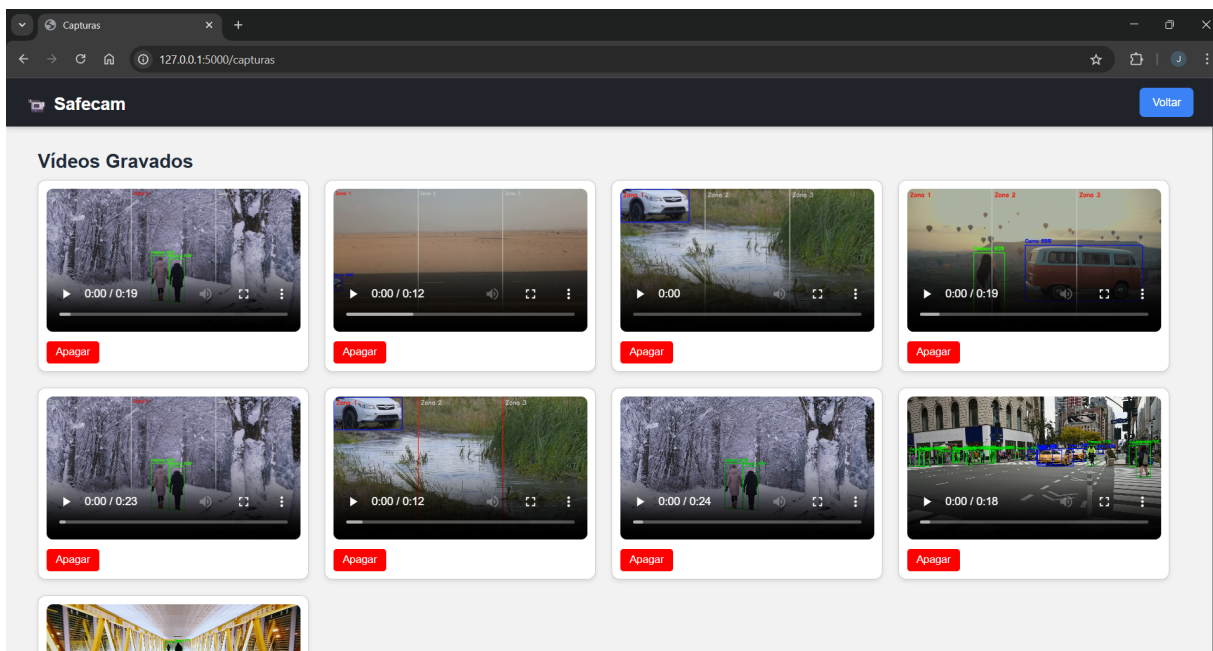


Figura 7: Página Capturas

3.6 Código desenvolvido

Este capítulo descreve os principais componentes do código implementado para o sistema de detecção e vigilância inteligente. A aplicação foi desenvolvida com base no modelo de detecção de objetos YOLO, integrando funcionalidades como anotação visual em tempo real, definição de zonas de interesse, gravação assíncrona de vídeo e envio automático de alertas por email. Todos os trechos de código apresentados foram retirados do ficheiro principal da aplicação (app.py).

3.6.1 Envio Automático de Alertas por Email

A função "send_email" é a função responsável por enviar emails com os alertas e vídeos em anexo. Utiliza o protocolo SMTP com autenticação e encriptação (TLS). É chamada automaticamente sempre que são detetados pessoas e/ou carros, ou no final de uma gravação para enviar o vídeo da captura.

```
def send_email(subject: str, body: str, attachments: list[str] = None):
    msg = MIMEMultipart()
    msg["Subject"] = subject
    msg["From"] = FROM_EMAIL
    msg["To"] = TO_EMAIL
    msg.attach(MIMEText(body, "plain"))
    if attachments:
        for filepath in attachments:
            if not os.path.isfile(filepath):
                continue
            part = MIMEBase("application", "octet-stream")
            with open(filepath, "rb") as f:
                part.set_payload(f.read())
            encoders.encode_base64(part)
            part.add_header(
                "Content-Disposition",
                f'attachment; filename="{os.path.basename(filepath)}"'
            )
            msg.attach(part)
    with smtplib.SMTP(SMTP_SERVER, SMTP_PORT) as server:
        server.starttls()
        server.login(SMTP_USER, SMTP_PASSWORD)
        server.send_message(msg)
```

Figura 8: Função para envio de Alertas por Email

3.6.2 Classificação e Anotação de Objetos Detetados

A função "gen_frames" é responsável por capturar, processar e devolver em sequência os frames de um vídeo, aplicando sobre eles a detecção de objetos em tempo real com base no modelo YOLO. Esta função constitui o núcleo do sistema de análise visual, permitindo que cada frame do vídeo seja analisado individualmente e que os objetos presentes sejam identificados, classificados e anotados visualmente.

Um dos blocos mais importantes da função "gen_frames" é o que trata da classificação e anotação dos objetos detetados. Este segmento percorre os resultados fornecidos pelo modelo YOLO, considerando apenas as detecções com confiança igual ou superior a 65%. São identificadas as classes dos objetos — neste caso, apenas pessoas (classe 0) e carros (classe 2) —, sendo desenhadas caixas delimitadoras sobre os frames com cores distintas (verde para pessoas e azul para carros), utilizando as funções "cv2.rectangle" e "cv2.putText". Cada anotação inclui

também uma etiqueta textual com o tipo de objeto e o nível de confiança em percentagem.

```
for box, cls_id, conf in zip(results.bboxes.xyxy, results.bboxes.cls, results.bboxes.conf):
    confidence = float(conf) * 100
    if confidence < 65:
        continue # ignora detecções fracas

    cls = int(cls_id)
    x1, y1, x2, y2 = map(int, box)

    if cls == 0:
        num_pessoas += 1
        tipo, cor = "Pessoa", (0, 255, 0)
    elif cls == 2:
        num_carros += 1
        tipo, cor = "Carro", (255, 0, 0)
    else:
        continue

    label = f"{tipo} {confidence:.0f}%"
    cv2.rectangle(frame, (x1, y1), (x2, y2), cor, 2)
    cv2.putText(frame, label, (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.6, cor, 2)
```

Figura 9: Código responsável pela classificação de pessoas e carros detetados

Outro bloco relevante da função é o que define e processa as zonas de deteção no vídeo. O frame é dividido em três zonas verticais (Zona 1, Zona 2 e Zona 3), com base na largura da imagem. Estas zonas são desenhadas graficamente com linhas divisórias e etiquetas visuais, permitindo determinar em que área do vídeo um objeto foi detetado. Quando um objeto entra numa destas zonas, o sistema destaca automaticamente essa área com uma cor de alerta, o que facilita a monitorização espacial da cena. A imagem seguinte ilustra o trecho de código responsável por esta lógica.

```

zonas = [zona1, zona2, zona3]
zona_alerta = [False, False, False]
for box, cls_id in zip(results.bboxes.xyxy, results.bboxes.cls):
    x1, y1, x2, y2 = map(int, box)
    for i, zona in enumerate(zonas):
        if verificar_zona(x1, y1, x2, y2, zona):
            zona_alerta[i] = True

x1_div = largura // 3
x2_div = 2 * largura // 3
cor_linha = (200, 200, 200)
cor1 = (0, 0, 255) if zona_alerta[0] else cor_linha
cor2 = (0, 0, 255) if zona_alerta[1] else cor_linha
cor3 = (0, 0, 255) if zona_alerta[2] else cor_linha

cv2.line(frame, (x1_div, 0), (x1_div, altura), cor_linha, 2)
cv2.line(frame, (x2_div, 0), (x2_div, altura), cor_linha, 2)

cv2.putText(frame, "Zona 1", (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, cor1, 2)
cv2.putText(frame, "Zona 2", (x1_div + 10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, cor2, 2)
cv2.putText(frame, "Zona 3", (x2_div + 10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, cor3, 2)

contador_texto = f"Pessoas: {num_pessoas} | Carros: {num_carros}"
(text_width, _), _ = cv2.getTextSize(contador_texto, cv2.FONT_HERSHEY_SIMPLEX, 1.0, 2)
x = frame.shape[1] - text_width - 10
y = frame.shape[0] - 10
cv2.putText(frame, contador_texto, (x, y), cv2.FONT_HERSHEY_SIMPLEX, 1.0, (0, 0, 255), 2)

```

Figura 10: Código responsável pela definição e visualização das zonas de detecção

O sistema integra uma lógica de emissão e término de alertas em tempo real. Quando são detetadas pessoas ou carros, é enviado um alerta por email e emitido um evento para a interface web através do SocketIO. Para evitar notificações excessivas, os alertas só são enviados na primeira detecção e são automaticamente terminados quando o objeto deixa de estar presente durante um intervalo de tempo de 2 segundos.

```
if num_pessoas > 0:
    if not alerta_pessoa:
        pending_alerts.append({'tipo': 'pessoa', 'message': f"{num_pessoas} pessoa(s) detetadas!"})
        send_email(subject="Safecam: Pessoa(s) Detetada(s)",
                    body=f"Detetámos {num_pessoas} pessoa(s) em {time.strftime('%Y-%m-%d %H:%M:%S')}".)
        alerta_pessoa = True
        tempo_pessoa = 0
    else:
        tempo_pessoa += 1
        if alerta_pessoa and tempo_pessoa >= TARGET_FPS * tempo_limite:
            pending_ends.append({'tipo': 'pessoa'})
            alerta_pessoa = False

if num_carros > 0:
    if not alerta_carro:
        pending_alerts.append({'tipo': 'carro', 'message': f"{num_carros} carro(s) detetados!"})
        send_email(subject="Safecam: Carro(s) Detetado(s)",
                    body=f"Detetámos {num_carros} carro(s) em {time.strftime('%Y-%m-%d %H:%M:%S')}".)
        alerta_carro = True
        tempo_carro = 0
    else:
        tempo_carro += 1
        if alerta_carro and tempo_carro >= TARGET_FPS * tempo_limite:
            pending_ends.append({'tipo': 'carro'})
            alerta_carro = False
```

Figura 11: Código responsável pela lógica de emissão e fim de alertas

3.7 Gravação Assíncrona de Vídeo Detetado

A gravação assíncrona de vídeo é realizada através da classe "VideoRecorder", que utiliza uma thread separada para armazenar os frames detetados sem interferir no processamento em tempo real. Os frames são colocados numa fila e gravados com resolução reduzida para poupar recursos. Quando o processo é finalizado, o vídeo é automaticamente encerrado e enviado por email como anexo do alerta. Esta abordagem permite manter a fluidez da análise visual enquanto os vídeos são gravados de forma eficiente em segundo plano.


```
class VideoRecorder(threading.Thread):
    def __init__(self, video_path, fps, size):
        super().__init__()
        self.video_path = video_path
        self.fps = fps
        self.original_size = size
        self.reduced_size = (size[0] // 2, size[1] // 2)
        self.queue = queue.Queue()
        self.stop_event = threading.Event()
        self.writer = cv2.VideoWriter(video_path, cv2.VideoWriter_fourcc(*'VP80'), fps, self.reduced_size)

    def run(self):
        while not self.stop_event.is_set() or not self.queue.empty():
            try:
                frame = self.queue.get(timeout=0.1)
                small_frame = cv2.resize(frame, self.reduced_size)
                self.writer.write(small_frame)
            except queue.Empty:
                continue
        self.writer.release()
        send_email(subject="Safecam: Vídeo de Alerta",
                  body="Segue em anexo o vídeo de alerta gravado.",
                  attachments=[self.video_path])
        print("[INFO] Gravação terminada.")
        global recorder
        recorder = None

    def add_frame(self, frame):
        self.queue.put(frame)

    def stop(self):
        self.stop_event.set()
```

Figura 12: Classe responsável pela gravação assíncrona dos vídeos detetados

4 Testes e Resultados

4.1 Cenários de Teste

Para validar o funcionamento do nosso protótipo, foram definidos e realizados vários cenários de teste, cada um com objetivos diferentes. Estes testes permitiram avaliar o desempenho do protótipo em diferentes condições de funcionamento.

Cada cenário é descrito com os objetivos, esquema de implementação, resultados obtidos e análise. Todas as principais funcionalidades foram verificadas, nomeadamente a detecção de objetos, alertas, gravação de capturas e envio de notificações por email.

Cenário	Objetivo do Teste
A	Testar o sistema em ambiente simulado com um vídeo pré-gravado.
B	Testar o sistema em ambiente real com a utilização da webcam.
C	Validar a contagem em tempo real de pessoas e carros e a sua localização em zonas específicas do enquadramento.

Tabela 3: Resumo dos Cenários de Teste

Os testes foram realizados num computador com processador Intel Core i5 7300HQ, 16 GB de RAM, sistema operativo Windows 10. O processamento do vídeo foi feito principalmente com recurso à CPU.

4.1.1 Cenário de teste A

- **Objetivos**

Avaliar o funcionamento do protótipo com base num vídeo pré-gravado, simulando um ambiente real com presença de pessoas. O objetivo é verificar se o sistema, utilizando o modelo YOLO11n, deteta corretamente os objetos e se os alertas e gravações são ativados como esperado.

- **Esquema**

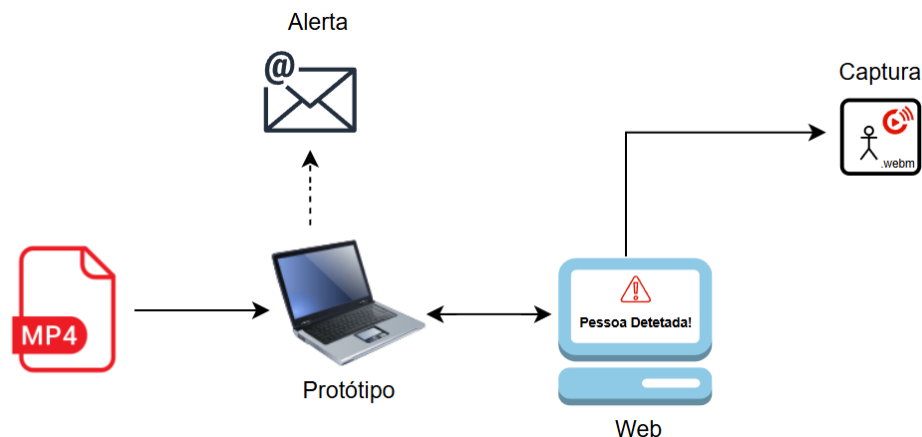


Figura 13: Cenário A

Neste cenário de teste, é utilizado um vídeo pré-gravado que substitui a entrada da web-cam. O protótipo analisa o vídeo utilizando o modelo YOLO11n. Sempre que o sistema identifica um destes objetos, é gerado um alerta em tempo real na interface WEB e enviado de imediato um email de notificação ao utilizador, indicando a presença de uma pessoa ou de um carro.

Simultaneamente, inicia-se a gravação de vídeo correspondente ao evento detetado. Quando o sistema deixa de detetar qualquer objeto durante um período de dois segundos, o alerta é automaticamente removido da interface, a gravação é finalizada, e um segundo email é enviado com o vídeo capturado em anexo.

• Resultados

Durante a reprodução do vídeo, o sistema detetou corretamente as pessoas (**fig.14**). Os alertas foram apresentados em tempo real na interface WEB (**fig.14**). Quando ocorreu uma deteção, foi enviado um email a informar o utilizador da ocorrência. Após o término da gravação do evento, foi enviado um segundo email com o vídeo gravado em anexo (**fig.15**). O vídeo foi armazenado na pasta de capturas e ficou disponível na interface capturas (**fig.16**) para visualização ou remoção.

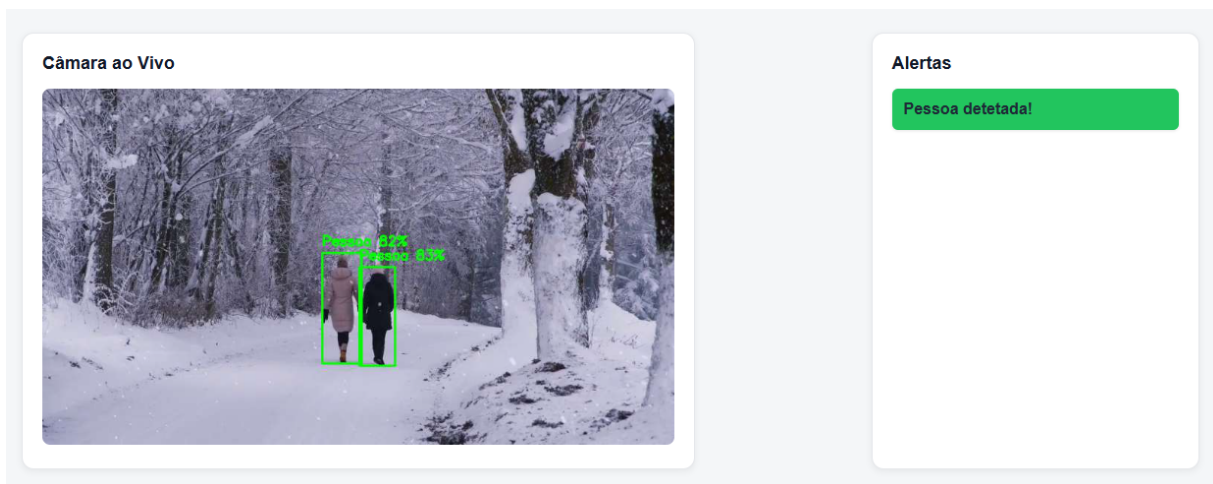


Figura 14: Detecção de pessoas e alertas na interface WEB

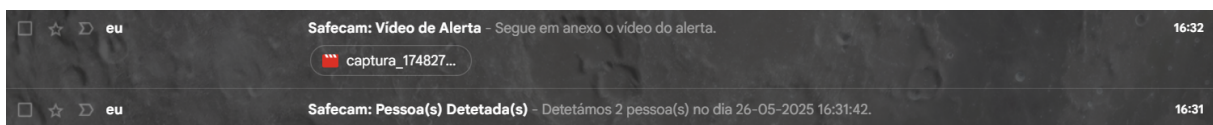


Figura 15: Alertas recebidos no email

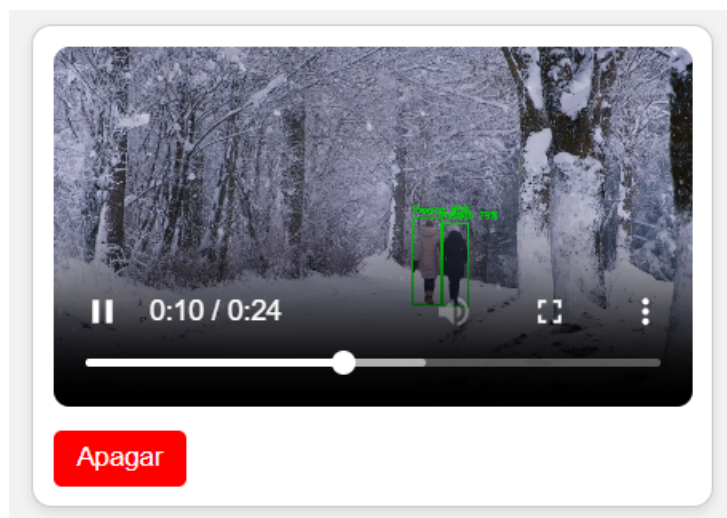


Figura 16: Vídeo gravado

- **Análise**

O sistema revelou-se funcional na análise de vídeos pré-gravados, conseguindo detetar corretamente pessoas e carros. No entanto, verificou-se que a substituição da webcam

por um vídeo com framerate elevado afeta o desempenho, aumentando o tempo de processamento. Apesar disso, os alertas foram emitidos corretamente na interface WEB em tempo real e as notificações por email funcionaram conforme esperado. O vídeo gravado foi armazenado localmente e enviado por email.

4.1.2 Cenário de teste B

- **Objetivos**

Avaliar o funcionamento do sistema em tempo real utilizando a webcam como fonte de entrada. Pretende-se verificar se o protótipo deteta corretamente pessoas e carros em situações reais e se os alertas, gravações e notificações por email funcionam conforme esperado.

- **Esquema**

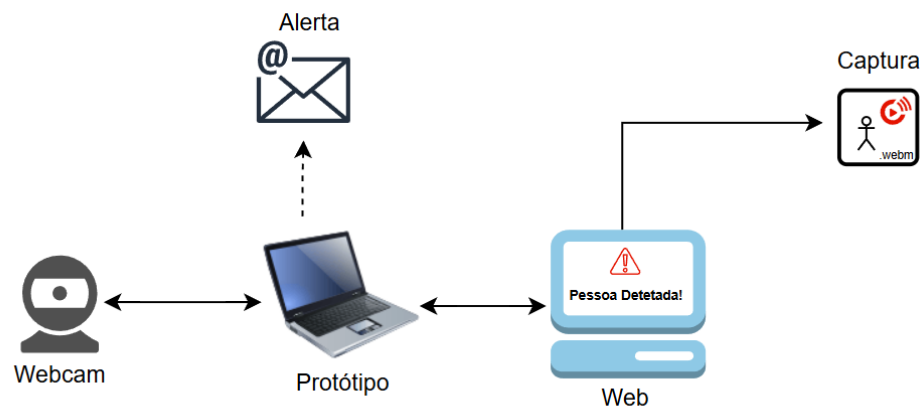


Figura 17: Cenário B

O utilizador acede ao sistema através da interface WEB. A webcam do dispositivo transmite vídeo em direto para o protótipo, que processa os frames continuamente. Quando uma pessoa ou um carro é detetado, é ativado um alerta visual na interface WEB e enviado um email ao utilizador com a notificação. Simultaneamente, inicia-se a gravação do vídeo. Quando deixam de ser detetados objetos durante 2 segundos, o alerta é removido e é enviado um segundo email com o vídeo capturado em anexo.

- **Resultados**

O sistema detetou corretamente pessoas e carros em tempo real através da webcam (como observado na **fig.18**). Os alertas foram exibidos de forma imediata na interface WEB, e os vídeos das ocorrências foram gravados e armazenados corretamente. Os emails de alerta foram enviados com sucesso.

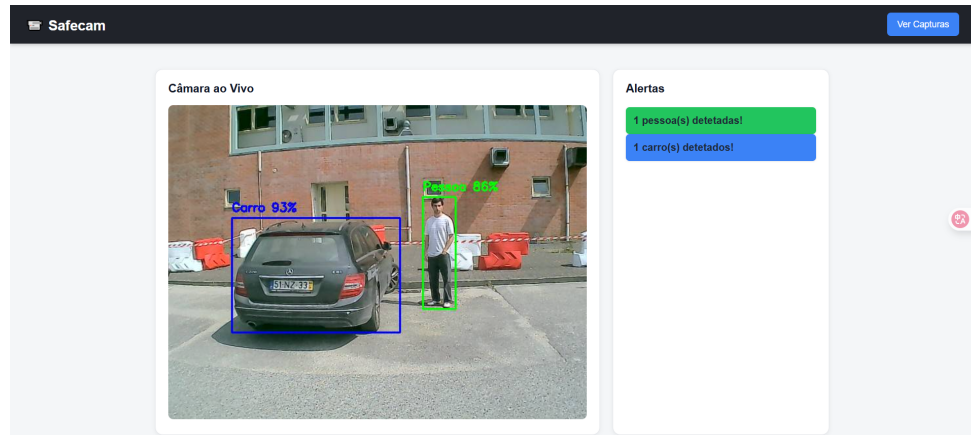


Figura 18: Detecção de pessoas e carros

- **Análise**

O teste demonstrou que o sistema é funcional em ambientes reais com webcam, conseguindo identificar objetos em tempo real sem qualquer problema no processamento. O sistema detetou corretamente as instâncias de pessoas e carros, ativando os alertas de forma precisa e emitindo notificações em tempo real. Além disso, foi enviado um email sempre que foi detetado um objeto e, após o término da gravação, o vídeo correspondente foi igualmente enviado por email com sucesso e guardado localmente na pasta capturas.

4.1.3 Cenário de teste C

- **Objetivos**

Avaliar a capacidade do sistema para contar o número de pessoas e carros detetados em tempo real e identificar a sua presença em zonas específicas do enquadramento do vídeo.

Pretende-se validar a precisão da contagem por zonas e a atualização dinâmica da interface com essa informação.

- **Esquema**

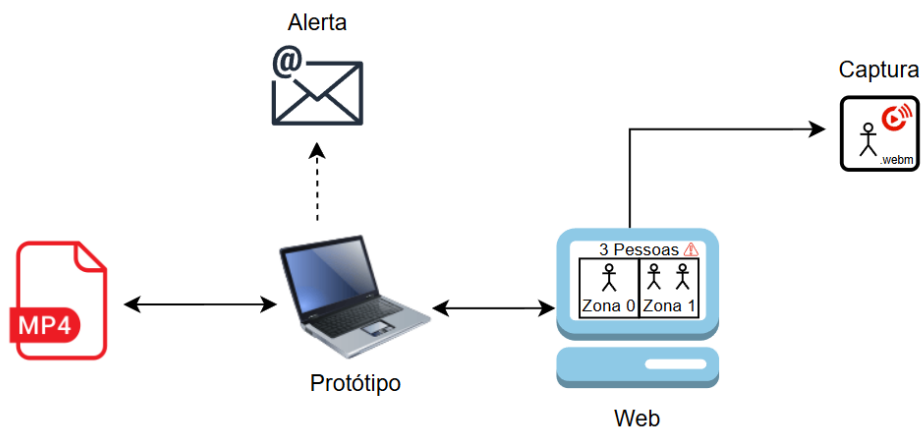


Figura 19: Cenário C

O sistema divide a imagem em regiões (por exemplo, zona 0, 1 e 2). Ao detetar pessoas/carros em cada região, o sistema apresenta a contagem total e a distribuição por zona, atualizando em tempo real.

- **Resultados**

Durante o teste, o sistema detetou corretamente as pessoas e os carros nas respetivas zonas da imagem. A contagem total foi atualizada corretamente no frontend e a distribuição por zonas correspondeu de forma precisa à posição dos objetos no enquadramento do vídeo (fig.20-21). Os emails de alerta foram enviados com sucesso.

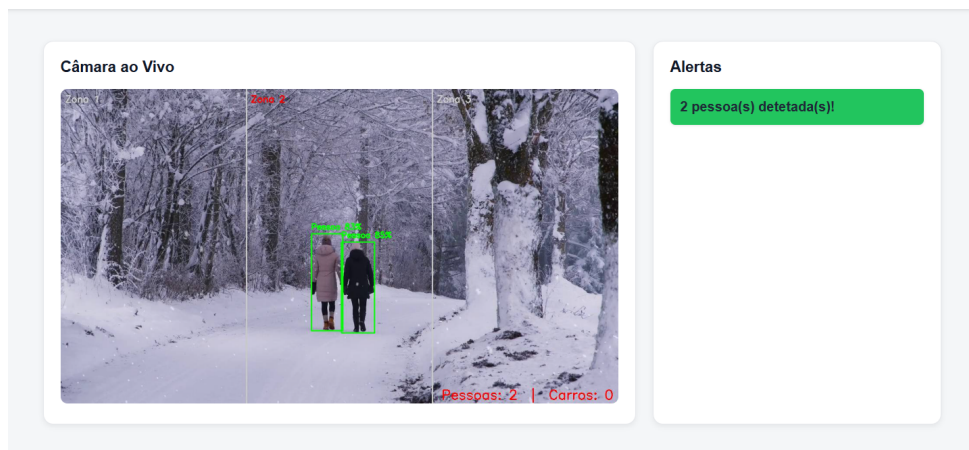


Figura 20: Detecção de pessoas

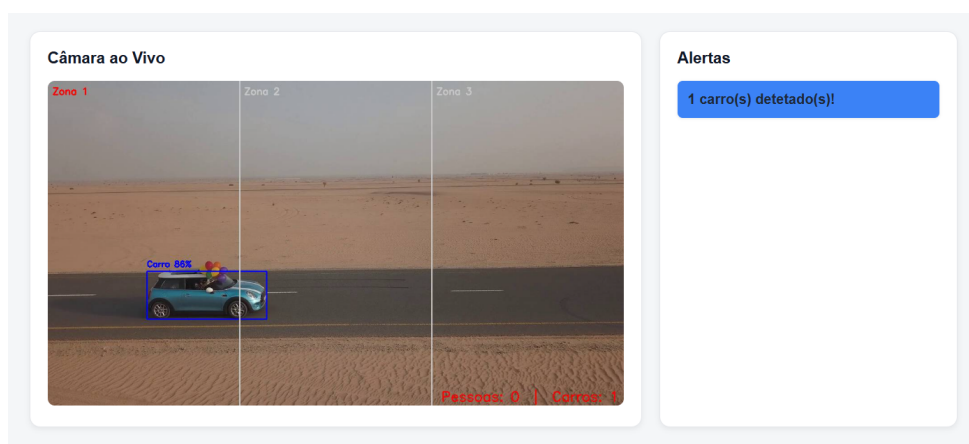


Figura 21: Detecção de carros

- **Análise**

O sistema demonstrou uma boa capacidade para realizar a contagem de pessoas e carros e atribuir a sua posição a zonas predefinidas do vídeo. Esta funcionalidade é útil para cenários como controlo de acessos ou ocupação de espaços. Com a integração desta funcionalidade aumenta a utilidade prática do protótipo.

5 Mais Valias

O SafeCam apresenta um conjunto de características que o distinguem de outras soluções de videovigilância convencionais:

- **Privacidade e controlo total sobre os dados**, ao funcionar numa infraestrutura de cloud privada, sem recurso a serviços externos.
- **Análise em tempo real**, com deteção automática de pessoas e veículos através de um modelo recente de inteligência artificial (YOLO11n).
- **Sistema de alertas configurável**, com notificações automáticas em caso de eventos relevantes.
- **Escalabilidade e facilidade de integração** com outros sistemas de segurança já existentes na organização.
- **Interface web centralizada e segura**, acessível apenas por utilizadores autorizados.

6 Análise crítica e proposta de melhorias

De forma geral, o desenvolvimento do projeto SafeCam decorreu de forma satisfatória, tendo sido possível atingir os objetivos inicialmente definidos. Todas as funcionalidades essenciais foram implementadas com sucesso, nomeadamente a deteção automática de pessoas e veículos, o envio de alertas por email e a visualização em tempo real através de uma interface web. O projeto permitiu aplicar e consolidar conhecimentos nas áreas de visão computacional, desenvolvimento web e integração de sistemas.

Apesar dos resultados positivos, existem diversas oportunidades de melhoria. Uma das principais propostas passa pela integração do SafeCam numa infraestrutura de cloud privada, permitindo uma gestão mais centralizada, segura e escalável dos dispositivos de videovigilância.

Outra melhoria relevante seria a melhoria do sistema de alertas, permitindo maior personalização, como filtros por tipo de deteção, zonas específicas da imagem, ou a utilização de diferentes canais de notificação (por exemplo, SMS ou integração com outras plataformas).

Adicionalmente, destaca-se a necessidade de melhorar a interoperabilidade entre diferentes dispositivos, garantindo que o sistema possa ser utilizado de forma consistente em equipamentos com diferentes características de hardware. Esta capacidade tornaria o SafeCam mais versátil e aplicável em contextos reais.

Estas propostas de evolução reforçam o potencial do SafeCam enquanto solução de videovigilância inteligente, segura e adaptável a diferentes cenários de utilização, sendo essenciais para que o projeto deixe de ser apenas um protótipo e possa evoluir para uma solução real e aplicável em contextos práticos.

7 Conclusão

Concluindo, o desenvolvimento da aplicação *SafeCam* foi bem-sucedido, tendo alcançado todos os objetivos propostos. A solução implementada permite a captação, detecção, transmissão e armazenamento de vídeo de forma eficiente e segura, recorrendo ao modelo YOLO11n para a detecção automática de pessoas e veículos em tempo real.

A interface web desenvolvida permite ao utilizador monitorizar o ambiente remotamente, visualizar os alertas em tempo real e aceder às gravações dos eventos, enquanto o sistema de notificações por e-mail assegura uma resposta imediata a ocorrências relevantes.

Os testes realizados em diferentes cenários validaram o correto funcionamento do sistema e a sua fiabilidade, demonstrando a utilidade prática do *SafeCam* em contextos de vigilância contínua. O trabalho cumpriu integralmente a proposta inicial, resultando numa solução leve, funcional e com potencial de evolução para integração em ambientes mais complexos.

Bibliografia

- [1] tecnoblog. Inteligência Artificial, 2025, <https://tecnoblog.net/responde/o-que-e-inteligencia-artificial/>
- [2] IBM. What is an AI model?, 2025, <https://www.ibm.com/think/topics/ai-model>
- [3] Ultralytics YOLO11, 2025, <https://docs.ultralytics.com/pt/models/yolo11/>
- [4] Frigate, 2025, <https://frigate.video/>
- [5] Deepstack, 2025, <https://deepstack.readthedocs.io/en/latest/object-detection/index.html>
- [6] Blue Iris, 2025, <https://blueirissoftware.com/>
- [7] ChatGPT. 2025, <https://chatgpt.com/>

Anexos

Em anexo segue o código desenvolvido e o vídeo demonstrativo das partes mais importantes do protótipo.

Código: https://myipleiria-my.sharepoint.com/:f:/g/personal/2222397_my_ipleiria_pt/Eo0q_Pfpei5OjVWXhG7Kt98BWYyWMckHDg2iOrnBaCj6jw?e=Blapxm

Vídeo: <https://www.youtube.com/watch?v=twR2gV4ViDI>